```
      create  app/assets/stylesheets/actiontext.css
      append  app/assets/stylesheets/application.tailwind.css
      create  app/views/active_storage/blobs/_blob.html.erb
      create  app/views/layouts/action_text/contents/_content.html.erb
Ensure image_processing gem has been enabled so image uploads will work
(remember to bundle!)
        gsub  Gemfile
       rails  railties:install:migrations FROM=active_storage,action_text
Copied migration 20221207145849_create_action_text_tables.action_text.rb
 from action_text
      invoke  test_unit
      create    test/fixtures/action_text/rich_texts.yml
```

You'll notice that the generator created a database migration. This is for the tables that Action Text uses to store the rich text itself.

Let's add those by running the `db:migrate` task:

```
> bin/rails db:migrate
== 20221207120454 CreateActionTextTables: migrating ======================
-- create_table(:action_text_rich_texts, {:id=>:primary_key})
   -> 0.0016s
== 20221207120454 CreateActionTextTables: migrated (0.0017s) =============
```

With all of that back-end setup out of the way, we can now make our UI. We will create this in the exact same way we've created other forms in our app, with the exception of the text area. Instead of using the `text_area` form helper to make a regular HTML `textarea` tag, we'll use `rich_text_area`, which will set up the Trix editor for us, enabling the UI part of Action Text.

Add this to `app/views/support_requests/index.html.erb`:

rails7/depot_tc/app/views/support_requests/index.html.erb

```
<ul>
  <% @support_requests.each do |support_request| %>
    <li>
      <h1>
        On <%= support_request.created_at.to_formatted_s(:long) %>
        <code><%= support_request.email %></code> writes:
      </h1>
      <p>
```